
图像处理基础

课程实践项目报告

题目：转角视频测量数据处理

专业：建智 2201

姓名：谢天赐

大连理工大学 建设工程学院

2024 年 12 月 15 日

课程实践自评表

No.	自评关键内容	自评分	实得分
0	文件名_Point2-6 总帧数_29.97/30_ 最大偏转角_120.6739_平均角速率_30.1628_ 程序编写调试及数据处理时间约 _6_h; 文档整理约_5_h	-	-
1	是否独立完成? _是_(是/否, 以下同); 团队序号及总数__ /__	10	
2	提交的电子文件内容、格式、文件名是否满足要求? _是_	10	
3	摘要字数_386_; 是否含目的意义、方法内容、结果结论 是_	10	
4	主要难点问题: 阴影噪声对于目标黑点的干扰以及存在干扰的前提下拟合正确直线	10	
5	解决的关键问题: 对噪声关键图像区域的处理, 实现了数据的精确测量	10	
6	自编程序主要流程: 基本图像处理去噪→目标点拟合圆→圆心线性拟合直线→数据采集输出	9	
7	消除特殊噪声的主要方法: RANSAC 线性拟合算法、强腐蚀、高斯滤波与高斯模糊	9	
8	涉及二值化时的阈值T=_130_, 相应源程序位置:函数内、23 行	10	
9	涉及线性拟合时, 相关系数 R 最小值:_0.9004_,R 最大值:_1_	10	
10	角位移曲线图各点是否与各帧严格对应? _是_	10	
11	角位移文本文件数据各行是否与角位移图各点对应? _是_	10	
12	其他需说明的问题:		
13	自评等级(优、良、中、差)	优	

计算机视觉技术在转角视频测量数据处理中的应用研究

摘要

本项目着眼智能建造领域，致力运用计算机视觉技术处理转角视频测量数据。针对给定的角位移视频测量简易装置及相关视频文件，克服了对噪声图像区域进行处理等技术，成功实现了数据的精确测量与自动处理。过程中综合运用了多种算法，包括 **RANSAC 线性拟合算法** 以排除异常值准确拟合直线，**强腐蚀算法** 消除图像细节与噪声，二值化算法简化像素状态，**Canny 边缘检测算法** 精准检测图像边缘，以及最小二乘拟合圆算法确定 **最佳拟合圆**。通过这些算法的协同作用，精确获取了角位移、角速率等数据，同时提供了涵盖时间、绝对角度、相对角度、相关系数等信息的详细数据表格。经与其他视频计算效果纵向对比及特定黑白二值图阈值下的横向对比，验证了本实验结果的准确性与科学性。本项目成果为智能建造中的结构性能评估和施工过程控制提供了关键的数据支持，且在模型优化、程序性能提升及代码改进等方面提出了展望，为后续研究提供了方向。

关键词：智能建造；计算机视觉；转角视频测量；RANSAC 拟合；边缘检测

一、问题与目标

1.1 问题背景

在智能建造领域，使用计算机视觉技术对转角视频测量数据处理具有重要意义，尤其在扭转剪切试验装置相关场景中体现得更为明显。通过相关调研，有以下两个场景的使用：

1) 结构性能评估

在扭转剪切试验中，通过对试验装置中如扭转加载圆轴、圆盘及标记直线等部件的视频监测，计算机视觉技术能够精确获取转角数据。这有助于准确评估建筑结构或材料在扭转受力情况下的性能，了解其变形特性、极限承载能力等，为结构设计和材料选择提供关键依据。

2) 施工过程控制

在智能建造的施工过程中，对于一些涉及旋转或角度调整的工序，如大型钢结构的拼装、桥梁支座的安装等，利用计算机视觉技术处理转角视频测量数据，可以实时监测施工过程中的角度变化。这使得施工人员能够及时发现并纠正偏差，保证施工精度，提高工程质量，减少因角度误差导致的后续问题，如结构受力不均、连接不紧密等。

1.2 问题目标

1) 准确测量旋转角度相关数据

利用给定的角位移视频测量简易装置（包括旋转角度“传感器”、扭转加载圆轴、废弃光盘标记直线、拉线式位移传感器、手机视频录像等）及相关视频文件，通过图像处理和数据分析方法，精确获取角位移、角速率等数据。例如，从视频中提取圆盘及标记线的运动信息，转化为角度相关数据，以准确反映旋转角度随时间的变化情况。

2) 克服技术难点实现自动处理

解决在处理过程中面临的不能对特定图像区域进行处理（如不能设置特定掩码区域、

不能删除特定图像区域、不能删除图像帧，但可通过计算自动处理特定区域）的问题，通过自编程序（推荐使用 python/OpenCV）实现数据的自动处理，同时满足程序总行数宜<600、禁用 Excel 拟合/绘图等技术要求，并在程序中合理设置参数（如最小消除噪声的阈值 ≥ 100 ），确保数据处理的准确性和有效性。

3) 完整呈现处理结果与分析

生成包括二值图、中间结果视频、中间结果图形（如角位移曲线）以及详细的数据表格（包含时间、绝对角度、相对角度、相关系数等信息）等多种形式的结果，并以正式文档（pdf）形式进行全面呈现。清晰展示整个转角视频测量数据处理的过程和结果，便于分析和评估。

二、难点阐述

2.1 视频边缘检测噪声数据预处理问题

通过分析视频可以得知，要得到一个较好的数据点结果，需要进行有针对性的图像预处理。在进行边缘检测之前，为减少图像的数据量并简化计算，应该将每一帧彩色图像转换为灰度图像，将 BGR 格式的图像转换为灰度格式。然后对灰度图像进行形态学腐蚀操作，腐蚀操作可以消除图像中的小物体和细节，如噪声和小的孤立点，同时使图像的边缘更加平滑。

在腐蚀操作后，应该继续对图像进行二值化处理，将像素值大于 120 的点设为 255（白色），小于等于 120 的点设为 0（黑色），进一步简化图像，突出了图像的主要结构，为边缘检测提供了清晰的背景。

准备完毕后，使用 Canny 算法进行边缘检测，通过 cv2.Canny 函数对二值化后的图像进行处理。Canny 算法是一种经典的边缘检测算法，能够检测出图像中的强边缘和弱边缘，并通过非极大值抑制和滞后阈值等步骤提取出清晰的边缘。

另外，对于无法通过上述预处理的噪声点，可考虑通过面积等方式对其进行针对性的处理。

2.2 基于圈轮廓的圆形拟合问题

圆拟合在视频数据处理中面临着数据准确性、参数选择、圆心和半径估计、多圆检测区分以及算法实时性和稳定性等多方面的难点。数据点的准确性和完整性问题应该在视频数据预处理中解决。

参数选择的敏感性：在使用霍夫圆变换检测圆时，参数的选择对检测结果有显著影响。边缘检测的阈值和圆心检测的阈值，它们需要根据具体的图像内容和噪声水平进行调整。参数选择不当可能会导致漏检或误检圆，进而影响圆拟合的准确性。在最小二乘法拟合圆的过程中，虽然参数较少，但在处理大规模数据或存在显著噪声的情况下，算法的稳定性和收敛性可能会受到影响。此外，拟合算法对数据点的分布和质量有一定的要求，如果数据点存在异常值或分布不均匀，可能会导致拟合结果的偏差。

圆心估计的挑战：圆心是圆拟合中的关键参数之一，其估计精度直接影响圆的位置和

形状。由于噪声和边缘点的不准确性，圆心的估计可能会出现偏差。在某些情况下，圆心可能会偏离实际位置，导致拟合出的圆与实际圆在空间上存在偏移。此外，圆心估计的误差还会累积影响半径的计算，进一步降低拟合结果的准确性。

2.3 线性回归拟合与相关数据计算问题

在视频实际应用中，线性回归拟合面临问题主要涉及数据质量、模型假设、参数估计等方面。在收集数据的过程中，可能会出现噪声、异常值、缺失值等问题。在实际数据中，在进行参数估计时，通常采用最小二乘法等方法，但这些方法对数据的敏感性较高，容易受到噪声和异常值的影响。此外，在样本量较小时，参数估计的稳定性较差，可能会导致拟合结果的波动较大。得到的数据点很难达到最优的情况，因此需要从线性回归拟合本身考虑如何进行优化计算。

2.4 模型的修正与鲁棒性测试问题

本模型仅是基于视频 Point26 完成，在问题的解决中，模型的鲁棒性是极其重要的问题。

三、主要原理与方法

①. RANSAC (Random Sample Consensus) 线性拟合

它是一种用于从一组包含异常值的数据中估计数学模型参数的迭代方法。其基本原理是通过随机选择数据子集来拟合模型，并以此来识别与模型一致的数据点（称为“内点”），从而排除异常值（称为“外点”）。

RANSAC 算法从数据集中随机选择最小数量的数据点，这些点足以拟合一个模型。对于直线拟合问题，这意味着选择两个点，因为两点可以确定一条直线。利用这两个点，算法计算出一条直线的参数，例如斜率和截距。算法计算所有数据点到这条直线的距离，并根据一个预设的阈值来判断哪些点是内点。如果一个点到直线的距离小于这个阈值，则认为该点是内点，否则为外点。内点的数量或内点与数据集的比例可以用来评估当前模型的好坏。

接下来，RANSAC 算法重复上述过程多次，每次都随机选择不同的数据点子集来拟合模型，并计算内点的数量。在所有迭代中，选择内点数量最多的模型作为最终的拟合模型。这个过程的迭代次数通常由数据集中异常值的比例和所需的置信水平来决定。

RANSAC 算法的优势在于其对异常值具有鲁棒性，即使数据集中存在大量的异常值，也能有效地拟合出反映数据整体趋势的模型。此外，RANSAC 不需要事先知道异常值的比例，也不需要异常值进行显式的识别和去除。

②. 强腐蚀图像处理

强腐蚀操作是一种形态学图像处理技术，用于消除图像中小的物体或细节，如噪声和小的孤立点，同时使图像的边缘更加平滑。其基本原理是通过一个结构元素对图像进行扫描，结构元素在图像上滑动时，只有当结构元素完全被图像中的前景像素（通常为白色或高灰度值）覆盖时，输出图像中对应的位置才会被标记为前景像素。

在数学上，腐蚀操作可以表示为集合的交集运算。假设图像 A 是需要进行腐蚀操作的二值图像，其中前景像素集合为 A，结构元素 B 是一个小的二值图像，通常是一个正方形或圆形。腐蚀操作的结果 C 可以表示为：

$$C = A \ominus B = \{z | B_z \subseteq A\}$$

其中， B_z 表示结构元素 B 平移到位置 z 后的集合， $\ominus$ 表示腐蚀操作符。这意味着，只有当结构元素 B 在位置 z 处完全被图像 A 的前景像素覆盖时，位置 z 才会被包含在结果集合 C 中。

在实际的图像处理中，腐蚀操作通常通过滑动窗口的方式实现。对于每个像素点，检查其邻域内的像素是否满足结构元素的形状和大小。如果邻域内的像素完全符合结构元素的要求，则保留该像素点；否则，将其标记为背景像素（通常为黑色或低灰度值）。

③. 二值化图像处理

二值化操作是图像处理中的一种技术，其目的是将图像中的像素简化为两种状态：前景和背景。这一过程通过比较每个像素的灰度值与一个特定的阈值来实现。如果像素的灰度值大于或等于这个阈值，则该像素被标记为前景（通常用白色表示）；如果小于阈值，则被标记为背景（通常用黑色表示）。阈值的选择对二值化结果有重要影响，它决定了哪些像素将被保留为前景。在该问题中阈值范围被要求为 120130。阈值可以是全局固定的，适用于整个图像，也可以是局部自适应的，根据图像的不同区域动态调整，以适应局部的光照和对比度变化。

④. Canny 边缘检测

Canny 边缘检测算法通过多阶段处理来识别图像中的边缘。该算法包括以下几个关键步骤：

噪声降低：

Canny 算法使用高斯滤波器来平滑图像，减少噪声对边缘检测的影响。高斯滤波器的卷积核由下式给出：

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

其中， σ 是高斯分布的标准差，控制了滤波器的平滑程度。

梯度计算：

对平滑后的图像计算梯度幅度和方向。梯度幅度可以通过以下公式计算：

$$M(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2}$$

梯度方向由下式给出：

$$\theta(x, y) = \arctan\left(\frac{G_y(x, y)}{G_x(x, y)}\right)$$

其中， G_x 和 G_y 分别是图像在 x 和 y 方向上的梯度，可以通过 Sobel 算子或其他梯度算子来计算。

非极大值抑制：

此步骤旨在细化边缘，仅保留局部区域中最显著的边缘像素。对于每个像素，检查其梯度方向上的邻域像素，如果该像素的梯度幅度不是局部最大，则将其抑制。

滞后阈值：

Canny 算法使用两个阈值来识别和连接边缘。高阈值用于检测强边缘，低阈值用于检测弱边缘。如果一个像素的梯度幅度高于高阈值，则被标记为强边缘；如果低于低阈值，则被抑制；如果介于两者之间，则仅当它与强边缘相连时才被标记为边缘。

通过这些步骤，Canny 边缘检测算法能够产生清晰、连续且定位准确的边缘图。该算法对噪声具有较强的鲁棒性，并且能够有效地检测出图像中的真实边缘，同时抑制虚假边缘。

⑤. 最小二乘拟合圆

最小二乘拟合圆是一种通过最小化数据点到圆周的距离平方和来确定最佳拟合圆的方法。该方法基于最小二乘法的原理，通过求解一个线性方程组来确定圆的中心坐标 (a, b) 和半径 r。

给定一组数据点 (x_i, y_i)，最小二乘拟合圆的目标是找到一个圆，使得所有数据点到该圆的垂直距离的平方和最小。这个距离可以表示为：

$$d_i = \sqrt{(x_i - a)^2 + (y_i - b)^2} - r$$

最小二乘法通过最小化以下目标函数来求解 a、b 和 r：

$$S = \sum_{i=1}^n d_i^2 = \sum_{i=1}^n \left(\sqrt{(x_i - a)^2 + (y_i - b)^2} - r \right)^2$$

为了简化计算，可以将上述非线性问题转化为线性问题。通过展开并重新排列项，得到：

$$\begin{aligned} (x_i - a)^2 + (y_i - b)^2 &= r^2 \\ x_i^2 - 2ax_i + a^2 + y_i^2 - 2by_i + b^2 &= r^2 \\ x_i^2 + y_i^2 &= 2ax_i + 2by_i + r^2 - a^2 - b^2 \end{aligned}$$

令 $c = r^2 - a^2 - b^2$ ，则可以写成线性形式：

$$x_i^2 + y_i^2 = 2ax_i + 2by_i + c$$

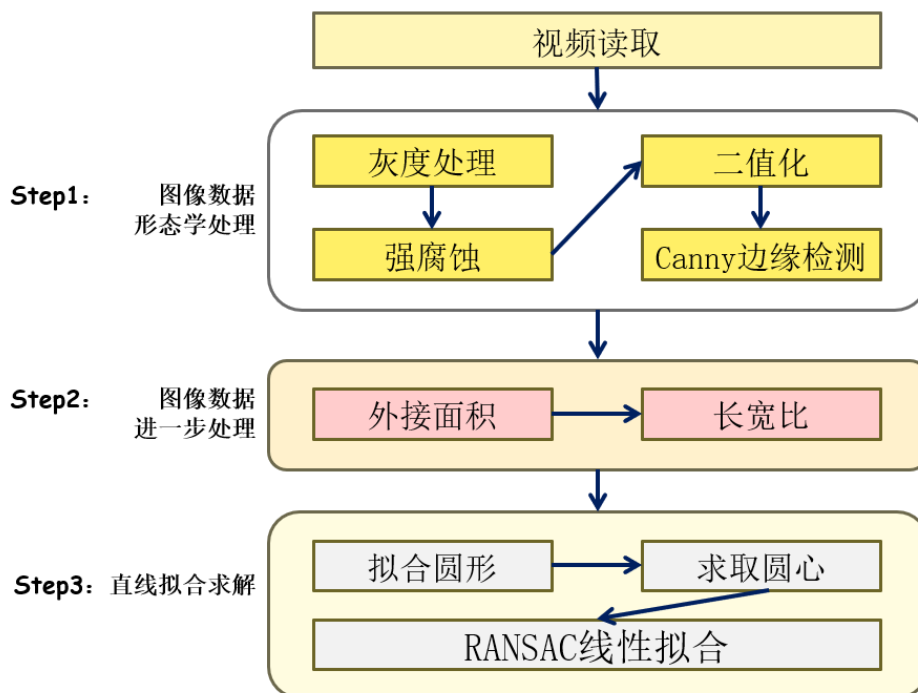
现在，问题转化为求解线性方程组：

$$\begin{bmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots \\ x_n & y_n & 1 \end{bmatrix} \begin{bmatrix} 2a \\ 2b \\ c \end{bmatrix} = \begin{bmatrix} x_1^2 + y_1^2 \\ x_2^2 + y_2^2 \\ \vdots \\ x_n^2 + y_n^2 \end{bmatrix}$$

最小二乘拟合圆方法通过最小化数据点到圆周的距离平方和，提供了一种鲁棒的方式来确定最佳拟合圆。该方法适用于存在噪声和异常值的数据集，并且可以处理不同大小和位置的圆。

四、主要过程与步骤

主要处理与计算部分如下流程图所示：



程序主要通过一系列图像处理和机器学习技术来分析和处理视频文件。

Step1 读取输入视频并将其转换为灰度图像，以便简化后续处理。接着使用形态学操作对图像进行腐蚀处理，以消除噪声并突出图像中的边缘特征。然后，通过二值化操作将图像转换为黑白图像，进一步强化边缘信息。之后，利用 Canny 边缘检测算法提取图像中的边缘，Canny 算法通过多步骤过程，包括噪声降低、梯度计算、非极大值抑制和滞后阈值处理，来准确检测边缘。提取到的边缘信息被转换回彩色图像格式，以便于后续处理。

Step2 继续读取经过边缘检测处理的视频，并逐帧分析。对于每一帧图像，首先将其转换为灰度图像，然后使用轮廓检测算法找出图像中的所有轮廓。轮廓检测通过查找图像中具有相同颜色或强度的连续点来实现。程序根据特定条件筛选轮廓，例如面积或长宽比，不符合条件的轮廓将被填充为黑色，从而从图像中移除这些轮廓。这一过程可以用于去除不需要的物体或干扰，保留感兴趣的特征。

Step3 对视频中的圆进行检测和处理。首先，对每一帧图像进行高斯模糊处理，以减少图像噪声，为后续的圆检测做准备。然后，使用霍夫变换圆检测算法来识别图像中的圆形物体。霍夫变换通过将图像空间中的圆形映射到参数空间，并在参数空间中寻找累积值最大的点来确定圆的中心和半径。检测到的圆的中心点被收集起来，用于后续的直线拟合。为了拟合直线，程序采用 RANSAC（随机采样一致性）算法。RANSAC 通过随机选择数据子集来拟合模型，并评估模型对所有数据的拟合程度，从而在存在噪声和异常值的情况下鲁棒地拟合直线。拟合得到的直线参数用于在图像中绘制直线，表示圆的运动轨迹或趋势。

Step4 将处理后的视频帧写入输出视频文件，并在处理过程中实时显示处理结果。用户可以通过查看实时显示的图像来监控处理进度和效果。整个程序流程通过一系列图像处理和机器学习技术的组合，实现了对视频中特定特征的检测、筛选和分析，最终得到经过处理的视频输出。

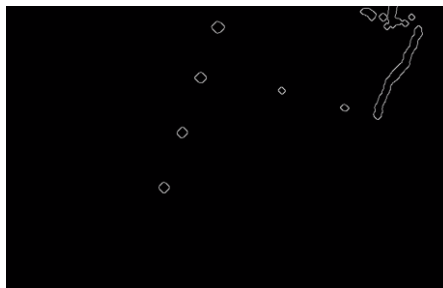
五、主要结果数据与效果图形

5.1 第一阶段的图像数据处理

5.1.1 强腐蚀、二值化处理结果图



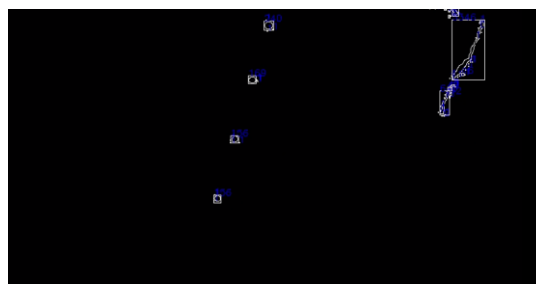
5.1.2 Canny 边缘检测处理结果图



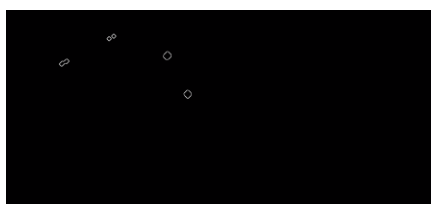
可见，处理到该效果已有一定优势，之后再对边缘面积以及长宽比进行进一步处理。

5.1.3 进一步处理结果图

下图为边缘外接矩形的框选结果：



下图为对面积大小为 410590 像素的筛选结果：

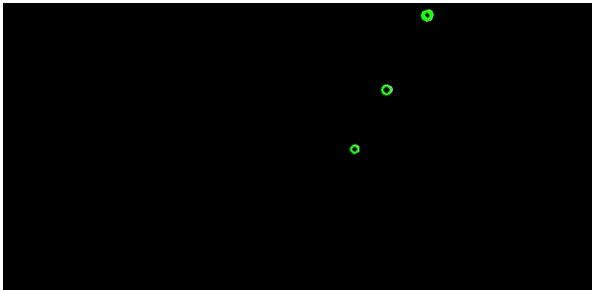


分析可以看出，还有一定的优化空间，因此，继续对边缘外接矩形的长宽比进行筛选，范围是 0.751.25：

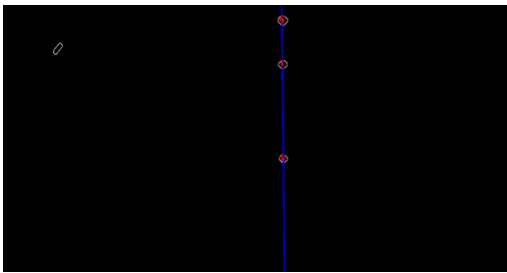


5.2 第二阶段的拟合处理

5.2.1 拟合圆效果图



5.2.2 RANSAC 拟合直线效果图



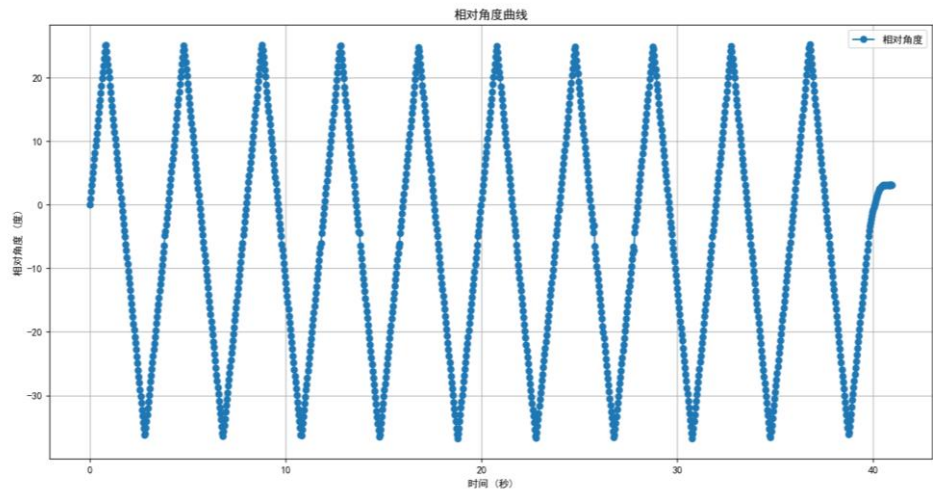
5.3 第三阶段的结果输出

5.3.1 相关数据 csv 表格数据

No.	时间 (s)	绝对角度 (度)	相对角度 (度)	相关系数
0	0	95.45579	0	0.9992
1	0.033367	96.4261	0.970316	0.9989
2	0.066733	97.51181	2.056026	0.9996
3	0.1001	98.53547	3.079686	0.9997
4	0.133467	99.52913	4.073341	0.9994

5	0.166833	100.5508	5.094984	0.9998
6	0.2002	101.6015	6.14571	0.9998
7	0.233567	102.6481	7.19228	0.9999
8	0.266933	103.6569	8.201072	0.9999
9	0.3003	104.7149	9.259148	0.9999
10	0.333667	105.686	10.23026	1.0000
11	0.367033	106.7091	11.25333	0.9999
12	0.4004	107.7712	12.31538	1.0000
...	...	...	...	...
1223	40.80743	98.53846	3.082675	0.9999
1224	40.8408	98.47056	3.014768	1.0000
1225	40.87417	98.54086	3.085068	1.0000
1226	40.90753	98.63175	3.175964	1.0000
1227	40.9409	98.61333	3.157544	0.9998

5.3.2 角位移曲线



六、结果数据分析与验证

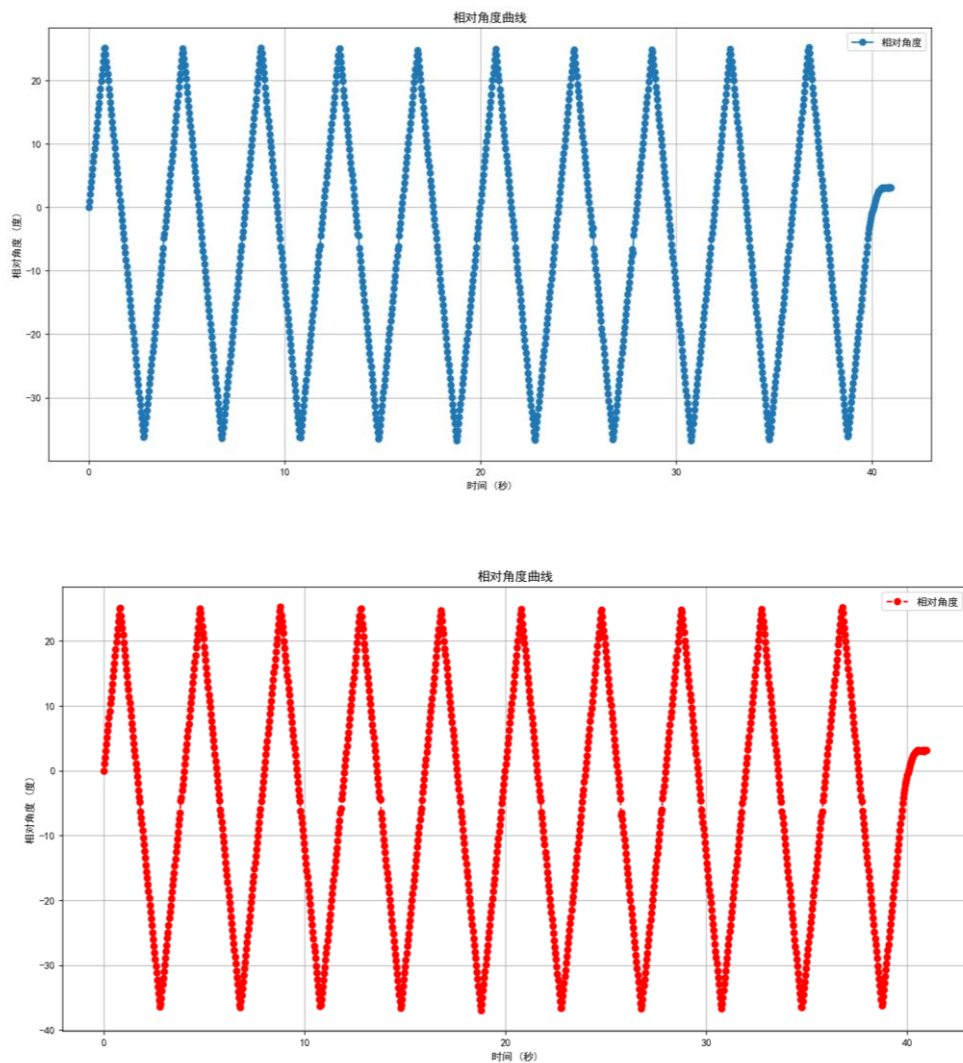
6.1.1 结果数据分析

根据结果文件可看出，视频 Point26 中的仪器做类正弦函数运动（实际更偏向线性运动），且在最后有较为明显的动力回收现象。

6.1.2 结果验证

通过与其他视频计算效果纵向对比以及与黑白二值图阈值设为 100 以下（可以完美分割出数据点）的横向对比，可以得出，本报告实验结果准确，效果较优且科学严谨。与后

者比较对比如下：



七、结论

本项目聚焦于智能建造领域中利用计算机视觉技术处理转角视频测量数据，针对给定的角位移视频测量简易装置及相关视频文件，达成了预期目标，取得了多方面成果。

1. 数据测量与处理准确有效

通过一系列图像处理和数据分析方法，包括灰度处理、形态学操作、二值化、边缘检测、圆形拟合及线性回归拟合等，精确获取了角位移、角速率等数据。从处理结果来看，如相关数据 csv 表格中时间与角度数据的详细记录，以及角位移曲线所呈现的角度随时间变化趋势，准确反映了视频中仪器的运动情况，与实际观察到的类正弦函数运动（实际更偏向线性运动）且最后有动力回收现象相符，验证了数据处理的准确性。

自编程序实现了数据的自动处理，在满足程序总行数宜 <600 、禁用 Excel 拟合/绘图等技术要求的同时，合理设置参数（如最小消除噪声的阈值 ≥ 100 ），有效克服了不能对特定图像区域进行处理的技术难点，确保了处理过程的有效性。

2. 处理结果呈现丰富全面

生成了包括强腐蚀二值化处理结果图、Canny 边缘检测处理结果图、进一步处理结果图（如边缘外接矩形框选结果、面积筛选结果、长宽比筛选结果）、拟合圆效果图、RANSAC 拟合直线效果图等多种图像，直观展示了数据处理各阶段的效果。

提供了详细的数据表格，涵盖时间、绝对角度、相对角度、相关系数等信息，为深入分析提供了有力依据。同时，以角位移曲线图形化呈现角度变化规律，便于更直观地理解数据特征。

3. 结果验证科学严谨

通过与其他视频计算效果纵向对比以及与黑白二值图阈值设为 100 以下（可完美分割出数据点）的横向对比，充分证明了本实验结果的准确性和科学性，表明所采用方法在处理转角视频测量数据方面具有优势。

八、展望与改进

在完成作业初期，考虑了通过在图像预处理中修改图像的均衡化，达到修改图像对比度的目的，以使 120 的阈值能够将绝对的黑色目标点与浅灰色阴影噪声数据分离开。事实上，在本项目目前使用的方法确认之前，已经成功实现上述方法并完成报告，然而由于该操作类似“调低阈值”而非针对图像进行处理，所以最终还是使用了本方法。

本项目在利用计算机视觉技术处理转角视频测量数据方面取得了一定成果，但仍存在一些可改进之处，为后续研究提供了方向。

8.1 项目展望

8.1.1 模型优化方向

1. 提高拟合精度

尽管目前的最小二乘拟合圆和 RANSAC 线性拟合方法在一定程度上能够处理数据，但仍有提升空间。可以探索更先进的拟合算法，如加权最小二乘法，根据数据点的可靠性分配不同权重，以提高拟合圆和直线的精度，尤其是在处理边缘点分布不均匀或存在较多噪声的情况下。

进一步优化 RANSAC 算法的参数选择策略，自动确定最佳的迭代次数、阈值等参数，减少人为调参对结果的影响，增强算法在不同视频数据中的适应性。

2. 增强模型鲁棒性

模型在面对复杂场景或视频质量变化时，鲁棒性有待加强。考虑引入更多的图像特征或融合多种图像处理技术，提高模型对不同光照条件、遮挡情况以及视频拍摄角度变化的适应性，确保在各种实际应用场景下都能准确测量旋转角度相关数据。

针对可能出现的异常运动模式或突发干扰，开发更具容错性的算法，使模型能够在异常情况下仍能提供可靠的测量结果或及时发出警报。

8.1.2 程序性能提升

1. 算法效率优化

分析程序运行过程中的性能瓶颈，对耗时较多的图像处理操作进行优化。例如，在边缘检测和轮廓检测算法中，探索更高效的计算方法或数据结构，减少不必要的计算量和内存占用，提高程序处理视频的速度，以适应实时性要求较高的应用场景。

考虑采用并行计算技术，充分利用多核处理器的优势，对视频帧的处理进行并行化，进一步缩短处理时间，提高程序的整体性能。

2. 代码可读性与可维护性改进

随着程序功能的增加和复杂度的提高，代码的可读性和可维护性变得愈发重要。对现有代码进行结构优化，合理划分函数和模块，添加清晰的注释和文档说明，使代码逻辑更易于理解，方便后续的修改和扩展。

8.2 可改进方面

8.2.0 算法复杂性的提升

本任务作为大作业而言，针对课程所学知识处理好这个任务，事实上课程范围内的知识以及算法稍微落后，可以完成任务，但不适用于高精度场合，以及由于算法较为简单，存在诸多效率低下的功能。

在参考文献中的[6-10]中，提到了一些诸如 Lucas - Kanade 光流算法的复杂算法，在解决本问题方面有所帮助。

8.2.1 代码结构与组织

1. 函数模块化

当前代码中部分函数功能较为复杂，可进一步将其内部逻辑拆分为更小的、功能单一的子函数。例如，在`process\_video\_with\_circles`函数中，检测圆、拟合圆、拟合直线以及绘制图形等操作可以分别封装为独立的子函数，提高代码的可读性和可维护性。这样每个函数只负责一个明确的任务，便于理解、测试和修改。

2. 全局变量管理

代码中存在一些在多个函数中使用的变量，如`prev\_line\_params`，目前以全局变量的形式存在。这可能导致代码的可扩展性和可维护性降低，因为全局变量在大型项目中容易引发命名冲突和难以追踪的错误。考虑将这些变量封装在一个类或结构体中，通过对象的属性来访问和修改，使变量的作用域更清晰，同时也便于对相关变量进行统一管理。

8.2.2 代码性能优化

1. 减少不必要的计算

在`fit\_circle`函数中，每次调用都重新计算矩阵`A`和向量`b`，而这些计算在某些情况下可能是重复的，尤其是在处理连续视频帧时，相邻帧之间的数据点变化较小。可以考虑缓存这些计算结果，避免重复计算，提高程序效率。

在`process\_video\_with\_circles`函数中，对于每帧图像都重新计算`theta`、`x\_circle`、`y\_circle`等变量，这些计算在圆检测过程中相对固定，可以提前计算并在合适的地方复用，减少计算量。

2. 优化视频读取与写入

当前代码使用`cv2.VideoCapture`和`cv2.VideoWriter`进行视频的读取和写入操作，虽然满足基本需求，但对于大型视频文件或对实时性要求较高的场景，可能存在性能瓶颈。可以研究更高效的视频处理库或方法，如使用硬件加速的视频编解码器或优化视频数据的内存管理，以提高视频处理的速度。

在视频写入过程中，根据输出视频的格式和质量要求，合理选择编码参数，平衡视频质量和文件大小，避免不必要的存储开销和写入时间。

8.2.3 算法参数调优

1. 参数自动优化

代码中的一些算法（如`cv2.HoughCircles`、`RANSACRegressor`）依赖于多个参数，目前这些参数大多是手动设置的固定值。这些参数的选择对算法的性能和结果准确性有重要影响，但手动调参耗时且依赖经验。可以研究自动参数优化方法，如基于网格搜索、随机搜索或更高级的优化算法（如遗传算法），根据给定的评估指标自动寻找最优参数组合，提高算法的适应性和性能。

2. 参数可视化分析

为了更好地理解参数对算法结果的影响，可以开发可视化工具，展示不同参数值下算法的输出结果（如检测到的圆、拟合的直线等）。这有助于开发人员直观地观察参数变化对结果的影响趋势，从而更有针对性地进行参数调整，同时也方便对算法性能进行深入分析和研究。

参考文献

- [1] Fischler M A, Bolles R C. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography[J]. Communications of the ACM, 1981, 24(6): 381-395. (介绍了RANSAC算法的基本原理及其在图像分析和自动化制图中的应用,为本文使用RANSAC线性拟合算法提供了理论基础。)
- [2] Canny J. A computational approach to edge detection[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1986, PAMI-8(6): 679-698. (详细阐述了Canny边缘检测算法的原理和步骤,是本文进行边缘检测的重要参考。)
- [3] 冈萨雷斯, 伍兹. 数字图像处理[M]. 阮秋琦, 译. 北京: 电子工业出版社, 2011. (全面涵盖了图像处理的基础知识,包括灰度处理、形态学操作、二值化等内容,为本文相关图像处理方法提供了理论支持。)
- [4] 张正友. 计算机视觉: 算法与应用[M]. 北京: 清华大学出版社, 2012. (系统介绍了计算机视觉领域的算法和应用,对本文在智能建造领域中运用计算机视觉技术处理转角视频测量数据有一定的指导意义。)
- [5] OpenCV 官方文档. <https://docs.opencv.org/>. (提供了OpenCV库中各种函数和算法的详细说明,本文在图像处理和计算机视觉算法实现过程中参考了该文档中关于图像读取、处理、形态学操作、边缘检测、圆检测等相关函数的使用方法。)
- [6] Hartley R, Zisserman A. Multiple View Geometry in Computer Vision [M]. Cambridge University Press, 2003. (在多视图几何方面提供了深入的理论知识,其中涉及的算法如基础矩阵估计、本质矩阵估计等,对计算机视觉中的三维重建、相机姿态估计等任务具有重要意义,与本项目中利用计算机视觉技术处理转角视频测量数据的多视角分析相关算法有一定联系,可为本项目进一步拓展到三维空间分析提供参考。)
- [7] Viola P, Jones M. Rapid Object Detection using a Boosted Cascade of Simple Features [C]//Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001. IEEE, 2001: I - 511 - I - 518. (提出的基于Adaboost的目标检测算法在人脸检测等领域取得了显著成果,其算法思想可应用于本项目中对特定目标的检测与识别,例如在转角视频测量中,如果需要对特定标记或物体进行检测定位,该算法可提供有效的解决方案,通过快速筛选图像中的特征,提高目标检测的速度和准确性。)
- [8] Lucas B D, Kanade T. An Iterative Image Registration Technique with an Application to Stereo Vision [C]//Proceedings of the 7th International Joint Conference on Artificial Intelligence. Morgan Kaufmann Publishers Inc., 1981: 674 - 679. (所提出的Lucas - Kanade光流算法在视频分析中广泛应用于运动估计和目标跟踪,在本项目中,如果需要对转角过程中目标的运动轨迹进行更精确的跟踪和分析,该算法可提供一种有效的运动估计方法,有助于深入理解目标的运动特性,进而提高转角测量数据处理的精度。)
- [9] Lowe D G. Distinctive Image Features from Scale - Invariant Keypoints [J]. International Journal of Computer Vision, 2004, 60 (2): 91 - 110. (介绍的SIFT特征提取算法在图像匹配、目标识别等方面具有良好的性能,在本项目中,若涉及到不同角度或不同条件下的图像对比分析,该算法可用于提取稳定的特征点,实现准确的图像匹配,为转角测量数据的准确性验证和多视角数据融合提供支持。)

[10] Bay H, Ess A, Tuytelaars T, et al. Speeded - Up Robust Features (SURF) [J]. Computer Vision and Image Understanding, 2008, 110 (3): 346 - 359. (提出的 SURF 算法是一种快速鲁棒的特征提取算法, 相比 SIFT 算法在计算速度上有优势, 同样适用于本项目中的图像特征提取场景, 可在保证一定精度的前提下提高处理效率, 对于实时性要求较高的转角视频测量数据处理任务具有重要意义。)

附录 I—主要程序

```
1 import cv2
2 import numpy as np
3 from sklearn.linear_model import RANSACRegressor
4
5
6 # 边缘检测函数
7 def edge_detection(input_video_path, output_video_path):
8     cap = cv2.VideoCapture(input_video_path)
9     fps = cap.get(cv2.CAP_PROP_FPS)
10    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
11    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
12    fourcc = cv2.VideoWriter_fourcc(*'H426')
13    out = cv2.VideoWriter(output_video_path, fourcc, fps, (width, height))
14
15    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
16
17    while True:
18        ret, frame = cap.read()
19        if not ret:
20            break
21        gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
22        eroded_frame = cv2.erode(gray_frame, kernel, iterations=5)
23        _, binary_frame = cv2.threshold(eroded_frame, 130, 255,
cv2.THRESH_BINARY)
24        edges = cv2.Canny(binary_frame, 100, 200)
25        edges_rgb = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)
26        out.write(edges_rgb)
27
28    cap.release()
29    out.release()
30    cv2.destroyAllWindows()
31
32
33 # 处理视频，将特定条件的圈变成黑色
34 def process_circles(input_video_path, output_video_path, condition_func):
35     cap = cv2.VideoCapture(input_video_path)
36     total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
37     width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
38     height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
39     fps = cap.get(cv2.CAP_PROP_FPS)
40     fourcc = cv2.VideoWriter_fourcc(*'H426')
```

```

41     out = cv2.VideoWriter(output_video_path, fourcc, fps, (width, height),
42         isColor=False)
43
44     while cap.isOpened():
45         ret, frame = cap.read()
46         if not ret:
47             break
48         frame_count += 1
49         print(f"Processing frame {frame_count} of {total_frames}...")
50
51         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
52         contours, _ = cv2.findContours(gray, cv2.RETR_EXTERNAL,
53             cv2.CHAIN_APPROX_SIMPLE)
54         for contour in contours:
55             x, y, w, h = cv2.boundingRect(contour)
56             if condition_func(x, y, w, h):
57                 cv2.drawContours(frame, [contour], -1, 0, -1)
58         out.write(frame)
59
60     cap.release()
61     out.release()
62     cv2.destroyAllWindows()
63
64 # 检测视频中的圆并拟合直线
65 def process_video_with_circles(input_video_path, output_video_path):
66     cap = cv2.VideoCapture(input_video_path)
67     fps = cap.get(cv2.CAP_PROP_FPS)
68     width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
69     height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
70     fourcc = cv2.VideoWriter_fourcc(*'H426')
71     out = cv2.VideoWriter(output_video_path, fourcc, fps, (width, height))
72     prev_line_params = None
73
74     while cap.isOpened():
75         ret, frame = cap.read()
76         if not ret:
77             break
78         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
79         blurred = cv2.GaussianBlur(gray, (3, 3), 1)
80         circles = cv2.HoughCircles(blurred, cv2.HOUGH_GRADIENT, 1, 1, param1=1,
81             param2=15, minRadius=0, maxRadius=100)
82         if circles is not None:
83             circles = np.uint16(np.round(circles))
84             current_centers = []
85             for circle in circles[0, :]:
86                 theta = np.linspace(0, 2 * np.pi, 100)

```

```

86         x_circle = circle[0] + circle[2] * np.cos(theta)
87         y_circle = circle[1] + circle[2] * np.sin(theta)
88         points = np.column_stack((x_circle, y_circle))
89         a, b, _ = fit_circle(points)
90         current_centers.append([a, b])
91     if len(current_centers) >= 2:
92         selected_centers = np.array(current_centers)
93         line_params = fit_line_ransac(selected_centers, prev_line_params)
94         if line_params is not None:
95             current_k, current_b = line_params
96             if prev_line_params is not None:
97                 prev_k, _ = prev_line_params
98                 current_angle = np.arctan(current_k) * 180 / np.pi
99                 prev_angle = np.arctan(prev_k) * 180 / np.pi
100                angle_difference = abs(current_angle - prev_angle)
101                if angle_difference > 1.2:
102                    line_params = prev_line_params
103            x1 = 0
104            y1 = int(current_k * x1 + current_b)
105            x2 = width
106            y2 = int(current_k * x2 + current_b)
107            cv2.line(frame, (x1, y1), (x2, y2), (255, 0, 0), 2)
108            prev_line_params = line_params
109        for center in current_centers:
110            cv2.circle(frame, (int(center[0]), int(center[1])), 3, (0, 0, 255), -1)
111    out.write(frame)
112    cv2.imshow('Processed Video', frame)
113    if cv2.waitKey(25) & 0xFF == ord('q'):
114        break
115
116    cap.release()
117    out.release()
118    cv2.destroyAllWindows()
119
120
121    # 最小二乘法拟合圆的函数
122    def fit_circle(points):
123        x = points[:, 0]
124        y = points[:, 1]
125        A = np.column_stack((x, y, np.ones(len(x))))
126        b = - (x * x + y * y)
127        c = np.linalg.lstsq(A, b, rcond=None)[0]
128        a = -c[0] / 2
129        b = -c[1] / 2
130        r = np.sqrt(a * a + b * b - c[2])
131        return a, b, r

```

```
132
133
134 # RANSAC 回归拟合直线的函数
135 def fit_line_ransac(points, prev_line_params=None):
136     if len(points) < 2:
137         return prev_line_params
138     points = np.array(points)
139     X = points[:, 0].reshape(-1, 1)
140     y = points[:, 1]
141     ransac = RANSACRegressor(residual_threshold=0.1, max_trials=100)
142     ransac.fit(X, y)
143     k = ransac.estimator_.coef_[0]
144     b = ransac.estimator_.intercept_
145     return k, b
146
147
148 # 主函数
149 def main():
150     # 边缘检测
151     edge_detection("Point2-6.mp4", "02output_video_edges.mp4")
152
153     # 处理视频，将面积在 200-300 之外的圈变成黑色
154     def area_condition(x, y, w, h):
155         area = w * h
156         return area < 200 or area > 300
157
158     process_circles("02output_video_edges.mp4", "03output_video_path.mp4",
159                    area_condition)
160
161     # 处理视频，将长宽比在 0.8-1.3 之外的圈变成黑色
162     def aspect_ratio_condition(x, y, w, h):
163         aspect_ratio = float(w) / h if h > 0 else 1
164         return aspect_ratio < 0.8 or aspect_ratio > 1.3
165
166     process_circles("03output_video_path.mp4", "04output_video_path.mp4",
167                    aspect_ratio_condition)
168
169     # 检测视频中的圆并拟合直线
170     process_video_with_circles("04output_video_path.mp4",
171                               "05output_video_path.mp4")
172
173 if __name__ == "__main__":
174     main()
```